

确定性 FA3 反向中的 dQ 依赖链调度优化

脱敏分享版 | 本文仅讨论可对外阐述的 GPU 调度机制、信号量依赖与验证方法，不对应任何特定模型或生产环境，也不表示相关 patch、仓库或交付物已公开。

结论先行

一次长序列、变长滑动窗口 Attention 的确定性反向回退，最终并不是算术吞吐或 dK/dV 引起的，而是 dQ 的 semaphore ticket 顺序与 reverse scheduler 不一致：后继 CTA 已经驻留，却在等待尚未被调度的 contributor，形成长时间 spin-wait。

优化没有增加 kernel、workspace 或设备到主机回传，而是在原 fused main kernel 内完成三项调整：使用 contributor-relative ticket、让 scheduler 与归约贡献顺序一致，并利用 host 端已有长度元数据对长短序列动态分流。代表性 packed trace 的完整 backward 从 6.755 ms 降至 1.699 ms，提升 3.98 倍；2K-12K 序列范围提升 1.59-5.37 倍，短序列与基线基本持平。目标路径连续 1000 次逐位一致，数值参考与通用测试共 3601/3601 通过。

这不是完整训练 step 的加速结论。训练框架短步端到端验证仍是生产启用前的必要门禁。

下文以机制图、伪代码和脱敏测量口径说明方法，不提供实际代码定位、补丁或私有实现的复现入口。

1. 问题背景：确定性为什么会变慢

高性能 Attention 反向通常允许多个 CTA 并发生成同一输出梯度的部分和。非确定性实现可依靠原子加法或不同到达顺序完成累加；确定性实现必须规定唯一的贡献顺序，确保相同输入在不同运行中得到相同 bit pattern。

在滑动窗口、sink token 和变长序列同时存在时，每个 query tile 对应的 key/value 覆盖范围并不相同。为了让 dQ 归约可重复，kernel 需要为贡献者建立 ticket，并通过 semaphore 控制写入顺序：

```
CTA contributor 0  └─┐
CTA contributor 1  └─┐→ ordered reduction → dQ tile
CTA contributor 2  └─┐
...                └─┘
```

确定性本身并不必然昂贵。真正的风险是：scheduler 的 CTA 出场顺序与 ticket 期待顺序不一致。

2. 先做因子隔离：不要把所有回退归因给 “deterministic”

完整 backward 同时包含 dQ、dK 和 dV。如果直接对整个 kernel 做 profile，很容易把等待、访存和计算混在一起。更可靠的方法是建立消融矩阵：

变体	dQ	dK/dV	semaphore 顺序	用途
基线	开	开	原生	总体对照
仅 dQ 确定性	开	关闭或固定	确定性	隔离 dQ 增量
仅 dK/dV 确定性	固定	开	确定性	隔离 dK/dV 增量
无等待控制	开	开	实验路径	判断是否为依赖链

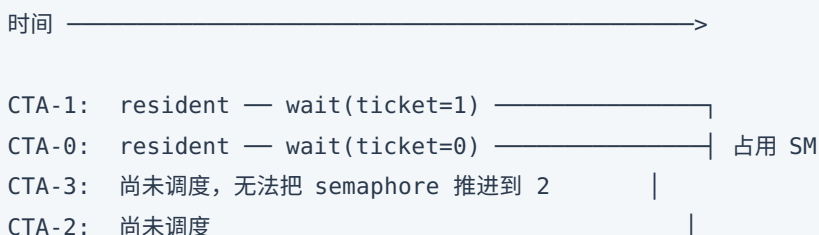
因子实验显示，dQ semaphore 路径占确定性增量的 99.67%，dK/dV 合计不足 0.4%。这一步排除了“矩阵计算不够快”“dK/dV 原子冲突”等直觉解释，把问题收敛到 dQ 的跨 CTA 顺序协议。

NCU 也给出了一组有力反证。启用问题 dQ 路径后，eligible warps 从 0.412 降到 0.052，issue active 从 28.23% 降到 4.28%，barrier stall 从 1.50 增至 41.53 cycles/issue；与此同时 DRAM 带宽从约 974.7 GB/s 降至 132.1 GB/s，TMA active 也明显下降。若瓶颈是带宽不够，DRAM/TMA 应更忙；实际情况恰好相反，说明 CTA 大部分时间在等待而不是搬运数据。

3. 根因：绝对 ticket 与 reverse scheduler 错位

3.1 一个简化例子

假设同一 dQ tile 需要四个 contributor，合法归约顺序为 $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$ 。reverse scheduler 为了平衡长短序列，可能先让 contributor 1 和 0 对应的 CTA 驻留：



等待中的 CTA 占着执行资源，真正能推进 ticket 的 CTA 反而更晚得到调度。这不是传统意义上的永久死锁，但会形成接近串行化的依赖链，长序列越明显。

3.2 为什么只改调度器还不够

如果 ticket 表示绝对 tile 编号，那么动态窗口、变长序列和 contributor 数变化都会改变“下一位是谁”。仅把 CTA 排序反转，不能保证所有 shape 的 ticket 关系都一致；更稳健的做法是让 ticket 表达“当前 dQ tile 内第几个有效 contributor”。

4. 设计：让协议表达相对贡献顺序

4.1 Contributor-relative ticket

对每个 dQ tile，先确定真正参与该 tile 的 contributor 集合，再以局部次序编号：

```
absolute tile id:      41, 37, 33, 29
relative contributor:  0,  1,  2,  3
```

semaphore 只关心局部 contributor 次序，不再依赖全局 tile 编号。这样 scheduler 只要按同一局部顺序派发，就不会让已驻留 CTA 等待一个尚未出现的绝对编号。

4.2 Reverse scheduler 对齐

长序列使用 reverse scheduler，让能解除更多依赖的 contributor 更早出场；短序列继续使用原生顺序，避免为较小工作量引入额外索引与分支成本。关键不是统一使用某一种顺序，而是 scheduler 与 semaphore 对“第几个贡献者”的定义完全一致。

4.3 Host metadata 动态分流

路径选择只使用 host 端本来就拥有的最大长度、平均长度等元数据：

```
if shape is short or dependency depth is small:
    native scheduler
else:
    contributor-relative reverse scheduler
```

因此不需要新 D2H、额外 workspace 或独立预处理 kernel，也不会在 CUDA Graph replay 中引入动态主机决策。

5. 实现约束

约束	处理方式
保持 fused main	不拆出新的归约 kernel
不增加 workspace	沿用现有 semaphore/元数据存储
不增加 D2H	只消费 host 已有长度信息
短序列不退化	不满足收益条件时走原生路径
确定性可解释	ticket 与 contributor 次序一一对应

这些约束很重要：如果通过额外 kernel 预排贡献顺序，即使局部等待减少，也可能被 launch、workspace 初始化和 Graph capture 成本抵消。

6. 验证设计

6.1 正确性不是只和旧输出逐位比较

改变合法累加顺序后，新结果可能与旧实现存在不同的舍入路径。因此验证分为三层：

- 与 FP32 reference 或高精度 oracle 做数值比较；
- 新路径相同输入重复运行，验证自身 bitwise deterministic；
- 扩展到通用 shape、窗口、长度分布和梯度回归。

“与旧实现逐位不同”不自动等于错误；“自身每次相同”也不能替代数值 reference。两类门禁必须同时存在。

6.2 测试结果

验证项	结果
目标 reverse 路径重复执行	1000 次 bitwise PASS
数值 reference 与通用回归	3601/3601 PASS
代表性 packed trace	完整 backward 3.98×
Uniform 2K–12K	1.59–5.37×
0.5K / 1K	+0.43% / -0.51%，基本持平

6.3 分层性能

层级	结果	可得出的结论
Main kernel	5.93×	依赖链等待被显著消除
代表性完整 backward	6.755 → 1.699 ms, 3.98×	优化能传递到算子级
训练框架 step	尚待短步 E2E	不能宣称完整训练收益

7. 容易犯的三个错误

错误一：看到长时间 kernel 就先改算术

如果时间线显示 CTA 在 semaphore 上自旋，继续优化 MMA、访存或 shared memory 不会解决主要问题。应先把 active compute 与 wait time 分开。

错误二：用更多并行度掩盖依赖顺序

增加 CTA 数可能让更多 CTA 同时进入等待，反而挤压真正能推进 semaphore 的 contributor。

错误三：把 kernel 结果写成训练 step 结果

算子级 3.98× 仍可能被其他 Attention、MLP、通信和框架开销稀释。生产口径必须等待真实训练框架的 Graph on/off、短步和长跑验证。

8. 可复用检查表

- ☐ 是否分别隔离 dQ、dK、dV 的确定性增量？
- ☐ semaphore 等待者与推进者的实际调度顺序是否可视化？
- ☐ ticket 表达的是绝对 tile，还是当前归约组内的相对 contributor？
- ☐ 新调度是否新增 kernel、workspace、D2H 或 capture-time 副作用？
- ☐ 是否同时验证数值 reference 与新路径自身 bitwise？
- ☐ 短序列是否有显式 fallback？
- ☐ 是否把 kernel、完整 backward 与训练 step 分开报告？

当前验证还发现，少数非目标边界组合的首次运行一致性仍未闭环，因此工程上已回退到保守路径。本文不展开对应业务 shape 或实现位置；该边界也不能被主测试矩阵掩盖。生产启用仍需覆盖训练框架短步、Graph on/off 和边界组合的长期稳定性。

9. 总结

确定性 GPU kernel 的瓶颈可能不是“做了更多计算”，而是“等待顺序与调度顺序互相打架”。把绝对 ticket 改成 contributor-relative 协议，并让 scheduler 与协议共享同一顺序定义，可以在不拆 kernel、不加 workspace 的情况下消除大部分自旋等待。更重要的是，性能修复必须和 reference、重复执行确定性、短尺寸 fallback 及训练框架门禁一起交付。